# CloudSage — Problems, Risks and Failure Modes

*Technical, AI, AWS, financial and operational challenges*

### 1. AWS Cost Attribution

AWS billing can contain shared/service-level costs and may not map perfectly to one resource. Resource-level attribution must be treated as approximate when the billing data does not support exact attribution.

### 2. Metrics Are Incomplete

CPU alone cannot justify rightsizing. Memory, network, latency, request rate, burst behavior and application health may matter.

### 3. Averages Hide Spikes

Use average plus P95/P99/max where relevant. A low average does not prove a resource is safe to downsize.

### 4. AI Hallucination

The LLM may invent facts or recommend an inappropriate change. Structured schemas, evidence-only context, deterministic validation and human approval reduce this risk.

### 5. AI Confidence Is Not Truth

A 95% model confidence field is not a statistical guarantee. It is only a model-generated signal. Backend policies must not treat it as proof.

### 6. Terraform Drift

Actual AWS configuration can differ from Terraform state/code. Drift must be detected before generating changes.

### 7. Resource Dependencies

A seemingly unused or expensive resource may support another service. Dependency checks and conservative policies are required.

### 8. Production Risk

Rightsizing or deleting resources can cause downtime, data loss or performance degradation. Production changes require approval and verification.

### 9. Savings Estimate Error

Predicted savings can differ from realized savings due to pricing, usage changes, commitments, shared costs and billing timing. Measure realized savings.

### 10. AWS API Limits

Large accounts can encounter throttling and pagination. Collectors need retries, backoff, pagination and caching.

### 11. Data Freshness

Cost data may lag. CloudWatch windows may be incomplete. Every analysis should include timestamps and data freshness.

### 12. Terraform Plan Surprises

A one-line intended change can cause unrelated changes. Parse the plan and enforce an allowed-impact policy.

### 13. Prompt Injection

Logs, tags, commit messages or other infrastructure strings can contain malicious instructions. Treat collected text as untrusted data and keep tool permissions outside the model.

### 14. Secrets

Never put AWS keys or LLM API keys into prompts, Git, images or logs. Use environment secrets locally and a proper secret manager in production.

### 15. Model/API Availability

The selected LLM provider can fail, rate-limit or change behavior. The application must fail safely and should not apply infrastructure when AI analysis is unavailable.

### 16. Cost of the Optimizer

The optimizer itself can incur LLM, database and AWS API costs. Track token usage and analysis cost.

### 17. Scope Explosion

Adding every AWS service, multi-account support, agents, UI and Kubernetes too early can make the project unfinishable. The version plan prevents this.